DAT101 Hovedoppgave 2026

Innledning

Hovedoppgaven for DAT101 ordinær eksamen fra vårsemesteret 2026 består av en oppgave.

Oppgaven er basert på pensum fra vår/høstsemesteret 2025/2026, og gjennomgang av praktisk programmering. Oppgaven har omtrent samme vanskelighetsgrad, men arbeidsmengden er forskjellig fra den praktiske workshopen som dette semesteret har hatt. Oppgaven beskriver hvor stor gruppe den er ment for, og dette vil det vektes for i sensuren av innleveringen. En gruppe på fire eller flere kandidater vil vektes strengere enn en gruppe på en til to kandidater.

Generelt for oppgaven, den blir gradert fra F til A i henhold til disse punktene:

- Applikasjonen/oppgaven må ikke være 100% løst for å levere eller bestå.
- Arbeidsmengden til gitt antall kandidater i en innlevering blir vurdert.
- Det legges ingen vekt på design eller stil.
- Hvilke elementer av oppgavespesifikasjonene er med.
- Koden trenger ikke være optimal.
- Dere må legge ved KI kommunikasjonen (eget PDF).
- Dere må kommentere hvor dere har brukt KI.
- Ved bruk av **Microsoft Copilot** eller annen IDE «Inline suggestions». Kodestrukturer som blir generert av KI og som strekker seg utover «pensum» i dette faget må kommenteres.

Grunnlag for underkjenning av innleveringen:

- Kodenotasjon avviker fra engelsk, det vil si det skal kun brukes engelske navn i objekter, klasser, funksjoner og variabler.
- Om det leveres flere identiske oppgaver fra to eller flere grupper, vil kommentarer til koden være avgjørende i sensuren. Og oppgaven vil bli vurdert som «en» innlevering.
- Mye bruk av KI uten at kandidaten viser forståelse av koden, vil bli underkjent.
- Innleveringen blir underkjent dersom **ZIP-filen** mangler korrekt mappestruktur, eller om vedlagt **PDF** med beskrivelse av arbeidsmetode ikke er inkludert.

Det er ingen begrensing på å legge klasser eller annen kode i eksterne Javascriptfiler (.js). Det vil være mest positivt for besvarelsen om flere script-filer blir brukt/laget.

Kommentarer i koden kan være på norsk om ønskelig. Ikke over-kommenter koden, det vil si ikke kommenter unødig, bruk kommentarer til å fortelle om eventuelle problemer, eller nødvendig informasjon (se vedlegg om kommentarer). **Det er viktig å kommentere kode som er delt med andre grupper. Forklar i detalj hva denne koden gjør!**

Innleveringen må skje på Inspera innen fristen og det leveres en ZIP-fil medfølgende innhold:

- **Samme struktur** som den som er vedlagt denne oppgaven i tillegg til egen kode.
- **PDF eller Word -dokument** som beskriver bruken av KI. Hvis ikke KI er brukt, beskriv dette i dokumentet.

Innholdsfortegnelse

Innledning 1
Innholdsfortegnelse 2
Nedalsting av filer og veileding til oppgavene 3
Oppgave 1 Snake 4
Beskrivelse 4
Beskrivelse av spilledesign 4
Slangen vokser 4
Menyene 5
Elementer som skal være med 5
Kode-kommentarer og notasjon 7
Kommentarer 7
Notasjon 8
Variabler 8
funksjoner 8
Objekter / Enum elementer 8
Klasser – attributter og metoder 9
forekomster 10

Nedlasting av filer og veiledning til oppgavene

Alle filene til oppgavene finner dere på Inspera, Canvas og GitHub. Her laster dere ned en zip-fil som inneholder hele mappestrukturen sammen med bibliotekene som følger med, uavhengig av hvilken oppgave dere velger. Det følger med noe kode til oppgaven. Koden som følger med, er noe annerledes enn den dere har programmert i workshopene. Men prinsippet er helt identisk slik vi har gjort i alle oppgavene til modul fem. Formålet med den medfølgende koden er, at dere skal vise evne i å lese eksisterende kode, og kunne bruke medfølgende biblioteker.

Når dere leverer oppgaven på Inspera må dere lage en gruppe. En av dere velger en gruppe og invitere de andre til gruppen med en nøkkel som personen får via Inspera. Om du er alene, så må du også opprette en gruppe, men da blir du eneste gruppemedlem. Om det ønskes individuell sensur internt i en gruppe, så må det spesifiseres og begrunnes i eget vedlagt dokument. Men fint om vi kan få beskjed på forhånd om det skulle dukke opp noe uforutsett i gruppen som kan ha konsekvenser for sensuren.

Faglærerne og assistentene kan veilede dere om dere står fast. Vi bistår i kode som ikke virker eller har problemer med. Og hjelper til med eventuelle løsninger. Men koden må skrives av kandidaten selv. Dere er tilgjengelige til å spørre om hjelp på Canvas, ved å sende beskjed til Arne-Thomas Aas Søndeled. Alle spørsmål om pensum, kode, eller annen oppgaverelevant innhold kan bli postet i plenum slik at alle kandidater sitter med samme tilgang til lik informasjon.

Husk hvordan spise en hel elefant!; Ikke prøv å løse hele dilemmaet på gang, gjør slik som vi har øvd på i workshopene. Analyser hva som skal gjøres å bryte ned i mindre deler, da blir det mye enklere å komme i gang.

Les oppgaven godt slik at dere ikke glemmer viktige elementer i løsningen deres.

Da er det bare å ønske dere lykke til alle sammen 😊

Oppgave Snake

Denne oppgaven er mest egnet til grupper med en til to kandidater.

Beskrivelse

Det skal programmeres et spill som heter «snake». Spillet handler om en «slange» som beveger seg over skjermen med en gitt fart. Det er ikke mulig å stoppe slangen i å bevege seg. Men den kan styres opp, ned, venstre og høyre. Tilfeldig rundt på brettet kommer det mat til slangen som slangen skal spise. Hver gang slangen spiser så blir den lengre og lengre. Slangen kan ikke kollidere med seg selv eller veggene i spillet, da dør den.

Spillet kan pauses med Space- tasten på tastaturet, og startes igjen ved å trykke på «Resume»-knappen.

wikipedia: https://en.wikipedia.org/wiki/Snake_(video_game_genre)

Beskrivelse av spilledesign

Eksempelkoden som følger med oppgaven **skal** det selvfølgelig bygges videre på. Det er allerede valgt et design i hvordan spillet fungerer.

Det er designet et usynlig rutenett med rader og kolonner. Hver celle i dette rutenett har litt informasjon pakket inn i en klasse «TBoardCellInfo». Og forekomster (Instances) av denne klassen er plassert i en tabell med rader og kolonner. Hvert element som ligger på spillebrettet (en celle), har en forekomst av «TBoardCell» som er den raden og den kolonnen som den befinner seg i.

Når slangen beveger seg over spillebrettet, så ender hodet til slangen retningen som den hadde når den passerte akkurat denne cellen. Da kan de andre elementene i slangen som kropp og hale lese hvilken retning hode hadde da den var på samme celle. Og vi får da effekten at kroppen «følger» etter hodet.

Når spillet starter så har de tre cellene (en hale, en kropp og ett hode) en «beskrivelse»: Her finnes det en slange-bit, hvis du/dere tester litt og lar slangen komme tilbake til denne posisjonen så blir det «game over». Slik kan dere sjekke om slangen prøver å spise seg selv. HUSK: Denne testen må fjernes før innleveringen.

HINT: Vi kan også bruke dette designet til «å si noe» om hva som befinner seg på hver celle. For eksempel om det er mat på akkurat denne cellen som hodet til slangen beveger seg over. Og slangen kan da «spise» maten.

Slangen vokser

For å få slangen til å vokse hver gang den konsumerer mat, kan man benytte følgende logikk: la hodet og alle kroppssegmenter, med unntak av halen, bevege seg ett steg frem. Dette skaper et "tomrom" hvor en ny kroppsdel kan plasseres. Det mest effektive her er å "kopiere" det bakerste segmentet av kroppen før det flyttes (ekskludert halen). På denne måten vil den nye instansen havne nøyaktig der den skal, og inneha den korrekte informasjonen som kreves for å følge den øvrige koden i neste spillsteg.

Menyene

Det er ikke laget noen menyer i den medfølgende koden, så dette må programmeres selv. Se i «png» filen hvilke knapper og menyer som finnes, og om noen av knappene har for eksempel animasjon.

Lag en komplett meny til spillet. Vær kreativ med å se hvilke elementer som sprite-sheet har.

Elementer som skal være med

Oppgaven kan ikke løses på andre måter enn å bygge videre på den koden som medfølger i oppgaven. Og det er viktig å følge oppgavebeskrivelsen slik at alle programmeringsteknikker og elementer i programmeringen er med.

a) Programmet skal ha et HTML definert canvas. Canvasets størrelse skal fastsettes i JavaScript-delen, samt eventuelle mus- eller tastatur- eventer skal programmeres i JavaScript.

b) Det skal vises poeng hele tiden under spillet, og poengene er avhengig av hvor lang tid det har tatt fra en matbit dukket opp til slangen har spist den. Jo kortere tid, jo høyere poengsum.

c) Slangens fart skal øke gradvis etter som den konsumerer mat.

d) Slangen skal bli lengre etter som den konsumerer mat.

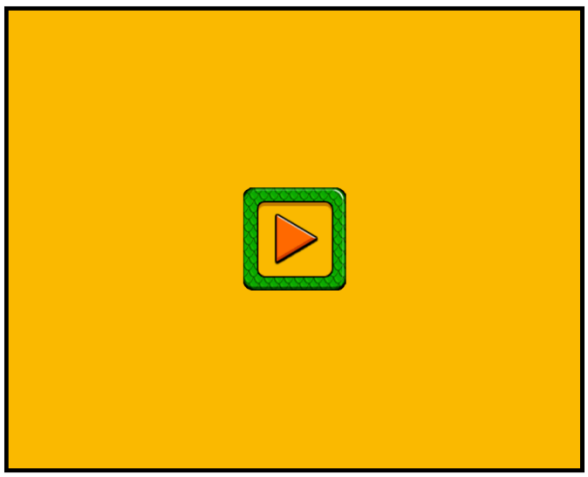
e) Spilleren skal ha mulighet til å starte, stoppe og pause spillet. Samt starte et nytt spill uten å oppdatere nettleseren.

f) Programmet skal ha «globale» variable, objekter og funksjoner. For å skape den «globale» følelsen så må disse eksporteres og importeres. Klassedefinerte objekter (Klasser), klassene skal ha både «Private» og «Public» medlemmer. Det premieres ekstra om «set» og «get»'ere blir brukt.

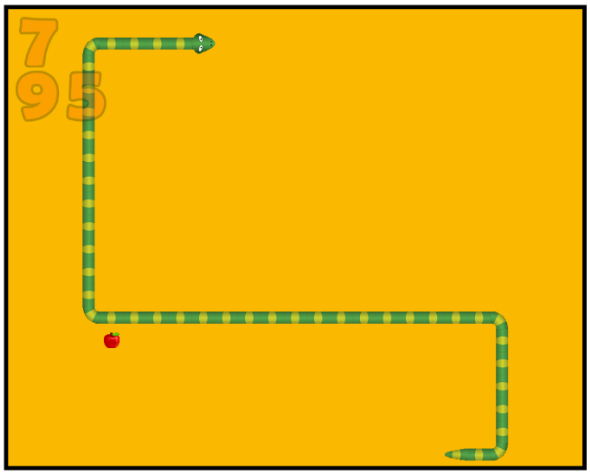
g) Programmet må inneholde enumererte statuser, løkker og logiske sjekker.

h) Klasse-struktur og navngivning står kandidaten/gruppen fritt til å velge selv, men det er viktig at notasjon og navngivning er entydig til objektets attributter og metoder. Samt alle navngitte variabler og konstanter skal være på engelsk.

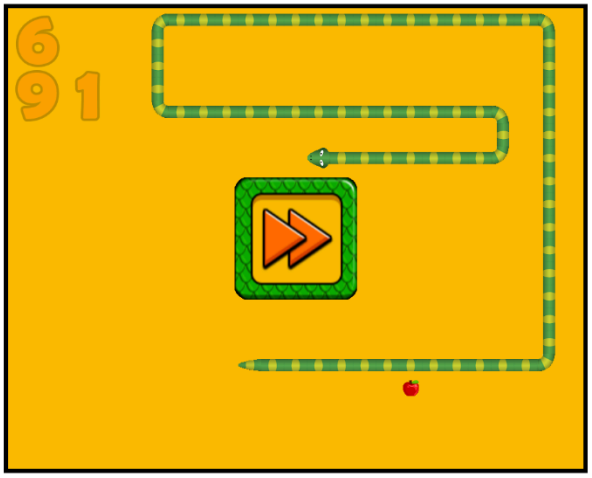
i) Slangens visuelle elementer skal bestå av sprites som lastes via et sprite-sheet.



Figur 1: Før spillet starter.



Figur 2: Mens spillet pågår



Figur 3: Spillet er pauset

Figur 4: Spillet er slutt

Kode-kommentarer og notasjon

Dette vedlegget beskriver litt om kommentarer og notasjon i koden du/dere skriver.

Kommentarer

Det har ingen betydning om du/dere kommenterer på norsk eller engelsk.

Det skal ikke overkommenteres i koden, det vil si: ikke skriv kommentarer som er uvesentlige eksempel:

```
//Dette er en for-løkke for rader
for(let row = 0; row < rows; row++){
  //Dette er en for-løkke for kolonner
  for(let col = 0; col < cols; col++){
    //En celle for hver rad og kolonne
   …
   …
   …
   …
  }
}
```

Det er ikke nødvendig med kommentarer som dette

Her er det greit med kommentarer slik at vi kan se hvilke knapper som er i bruk. For eksempel kode 38 sier ikke så my om knappen men `//Up key` forteller hvilken knapp som brukes. I slike tilfeller er det best å bruke enum-objekter.

```
function keyDown(e) {
  const code = e.keyCode;
  switch (code) {
    case 38: //Up key
      …
      break;
    default: //Everything else
      break;
  }
}
```

```
function moveHero(){
  // Need to check speed in order to NOT get hero "glued" to the bottom
  if(hero.hitBottom() && (this.hero.speed.y>0) ) {
    //if speed is positive and hero has passed the bottom of the canvas
    // set speed and gravity to zero in order to stop her from moving down
    hero.speed.y = 0;
    hero.gravity = 0;
  }
  if(this.gameOver === false) {
    hero.move();
  }
  hero.draw();
}
```

Dette er også bra kommentarer, ikke uvesentlige og de er beskrivende for funksjonaliteten til spillet.

Notasjon

All notasjon skal foregå på engelsk. Det vil si, ikke bruk norsk språk i klasser, objekter, forekomster, variabler eller funksjoner. Navnet som du bruker, må ha en tilhørighet til hva den **er** eller **gjør**.

Variabler

Start variablene med små bokstaver og om navnet består av flere ord skill ordene fra hverandre med stor bokstav.

```
let cvs = null;
let ctx = null;
let canvasFrame = null;
let game = null;
let interval = null;
let frameNumber = 1;
```

Funksjoner

Start funksjonsnavn med små eller store bokstaver, men vær konsekvent. Noen foretrekker å ha storbokstav i starten på en funksjon for å skille variabelnavn og funksjonsnavn, jeg foretrekker å bruke liten bokstav i starten på både funksjoner og variabler.

```
function getRandom(min , max){
  return Math.ceil(Math.random()*(max-min))+min;
}
```

Objekter / Enum elementer

Objekter som aldri endrer seg eller som brukes som «enum» elementer starter med stor bokstav. Alle attributtene til objektet har også stor bokstav. Hvis objektet blir brukt som «enum» element er det lurt å starte med stor E i navnet.

```
const EButtonState = {
  Up:0,
  Down:1
};

const EGameObjectType = {
  Hero: 1,
  Obstacle: 2
};
```

Enum objekter med stor «E»

```
const Coordinates = {
  UserColorPic: [
  //Right side user color picker
  {x: 262, y: 87},      //Black
  {x: 262, y: 131},     //Blue
  {x: 262, y: 175},     //Brown
  {x: 262, y: 219},     //Green
```

Statiske objekt som aldri endrer verdi mens programmet kjører.

```
  {x: 262, y: 263},      //Orange
  {x: 262, y: 307},      //Red
  {x: 262, y: 351},      //White
  {x: 262, y: 396}],     //Yellow
  CorrectAnswer: [
    //Computer correct answer
    {x: 72, y: 46}, {x: 102, y: 46}, {x: 132, y: 46}, {x: 162, y: 46}],
};
```

Klasser – attributter og metoder

En klasse starter med en stor bokstav, dere velger selv deres bokstav men dere er jo nå godt kjent med «T» og står fritt til å bruke den. Alle attributtene og metodene starter med små bokstaver. Om de er «Public» eller «Private» spiller ingen rolle for navngivningen.

```
class TRect{
  constructor(aX,aY,aWidth,aHeight){
    this.left = aX;
    this.top = aY;
    this.width = aWidth;
    this.height = aHeight;
    this.right = this.left + this.width;
    this.bottom = this.top + this.height;
  }

  move(aDeltaX, aDeltaY){
    this.left += aDeltaX;
    this.top += aDeltaY;
    this.bottom = this.top + this.height;
    this.right = this.left + this.width;
  };

  setPos(aCenterPos){
    this.left = aCenterPos.x - (this.width/2);
    this.top = aCenterPos.y - (this.height/2);
    this.bottom = this.top + this.height;
    this.right = this.left + this.width;
  };

  mouseHit(aPos) {
    return !(( aPos.x<this.left) || (aPos.x>this.right) ||
             (aPos.y<this.top)  || (aPos.y>this.bottom));
  };
}
```

FOREKOMSTER

Forekomster av en klasse starter med småbokstaver på lik linje med variabler, de skiller seg ut blant statiske objekter på grunn av at de endrer innhold etter som programmet går. Men navngivning er lik.

```
const surviveGame = new TGame();

let canvasFrame = null;
canvasFrame = new TRect(0,0,cvs.width, cvs.height);
canvasFrame.thick = 10;
```